SIS #02 — AI-Assisted UML Modeling: University Course Registration

2 points · individual · AI use **required** (Level D) · no application code · deadline **Sunday 18 October 2026, 23:59** (Almaty)

The submission is the link to your open pull request, pasted into the Teams assignment. No file uploads. Unzip SIS_02.zip into the root of your se-practice repository, on a new branch sis-02. The full assignment is sis-02/README.md; this handout is the short version.

Your choice, frozen once made. Any AI assistant you have access to; one is enough for all four prompts. Record the **exact model** it shows — "ChatGPT" is a tool, not a model. If the tool does not show the model, write `not displayed`.

Identical for everyone: the scenario, rules R1–R5, the four prompts word for word, cases T1–T4, the word limits, the class names `Student`, `Course`, `CourseOffering`, `Enrollment`, and the file names. Both diagrams are **PlantUML**.

1 The system

A student browses an electronic course catalog, chooses a course offering and requests registration. Each offering has a limited number of places. A **Course** describes a subject. A **CourseOffering** is one scheduled section in one semester. Registration always targets a specific offering. Assume that the student is signed in, the selected offering exists, and its capacity is a positive whole number.

Rule	What the system must do
R1. Catalog	Show the course code, title, offering and available places for one semester.
R2. Capacity	Recheck current availability when registering. Enrollment must never exceed the offering's capacity.
R3. Duplicates	A student may have at most one enrollment in the same offering. Reject a repeat request.
R4. Success	Create exactly one Enrollment linking the student and offering, update availability and return a confirmation.
R5. Rejection	Explain "course full" or "already enrolled". Create no enrollment and leave the enrollment count unchanged.

A catalog view does not reserve a place. Model the final checks and enrollment creation as **one atomic operation**, so competing requests cannot take the same last place. A note describing it is enough. **Out of scope:** payments, prerequisites, timetable clashes, waiting lists, cancellation. Use fictional students and courses. Treat any extra AI suggestion as a proposal you accept or reject explicitly.

2 The work, in order

Stage	You	The assistant	Goes to
Before AI	Write a **Scope:** statement (15 words or more) and at least two initial assumptions. **Commit and push.** Never edit that section again.	—	`report.md` §10
A. Requirements	Review all six criteria, map each to R1–R5, remove anything out of scope.	Prompt A: 3 stories (US-01–03), 6 Given / When / Then criteria (AC-01–06)	§11 B1 → §1, §2
B. Modeling	Save both PlantUML blocks **unedited**, render and inspect them.	Prompt B: a sequence diagram and a class diagram	§11 B2 · `models/original/` · §12
C. Critique	Judge every point yourself: accept, reject or modify. The critique is a claim, not a verdict.	Prompt C, in a **new chat**: checks both models against the rules	§11 B3 → §7
D. Revision	Read the revision, edit it where it is still wrong, save the final files, render them.	Prompt D: applies **your** accepted decisions	§11 B4 · `models/*.puml` · §3, §4
Validate	Walk T1–T4 through your final sequence diagram. Trace every rule. Record at least 3 review decisions.	—	§5, §6, §7
Write	Modeling explanation **200–300 words**. Reflection **150–200 words**, in your own words. Two source checks with exact locators: slide, page, section or figure, or a URL with an access date.	—	§8, §9, §13

Three commits minimum, authored by you. This is the order we expect to see:

```
git commit -m "sis-02: scope and initial assumptions, before Prompt A"
git commit -m "sis-02: approved requirements and the AI's original diagrams"
git commit -m "sis-02: final models, walkthroughs, review and report"
git commit -m "sis-02: declaration"
```

3 The four prompts — word for word

Replace every `[bracketed field]`, change nothing else, run them in order. Save the complete prompt you sent and the complete response in `report.md` §11 before editing anything. Copy the prompts from `README.md` §4; the text below is the same.

Prompt A — requirements. `[paste them]` = the scenario and R1–R5. `[paste]` = your §10 assumptions.

```
Act as a requirements analyst. Use only this course registration scenario and rules R1–R5: [paste them]. My initial
assumptions are [paste].
Draft 3 user stories and 2 Given / When / Then acceptance criteria for each. Use IDs US-01 to US-03 and AC-01 to AC-06. Map
each criterion to rule IDs.
Cover normal and rejection behavior. List uncertainty separately. Do not add excluded features or invent university
policies.
```

Prompt B — UML drafts. Under it, paste your approved §2: all three stories and all six criteria, with their IDs.

```
Using my approved scenario, R1–R5, stories and criteria below, return separate PlantUML code blocks for a sequence diagram
and a domain class diagram.
Sequence: include catalog browsing and guarded success, full-course and duplicate outcomes. Model final checks and creation
as one atomic operation.
Class: show domain concepts, useful attributes and multiplicities. State the capacity and uniqueness constraints. Explain
assumptions and any technical lifelines.
```

Prompt C — model critique. New chat. Under it, paste both original PlantUML blocks and your acceptance criteria.

```
Review these two UML models against R1–R5 and my acceptance criteria. Check message order, guards, state changes,
multiplicities and naming.
Inspect the four validation cases: available places, exactly one place, full course after a stale catalog view, and
duplicate registration.
For each issue, cite the affected rule and model element. Explain the impact and propose a focused correction. Separate
genuine defects from optional design choices.
```

Prompt D — focused revision. `Original sources: [paste]` = the two files in `models/original/`, in full. Accepted nothing? Write "no changes accepted" and the reason in `[list]`; it still counts as your fourth exchange.

```
Revise the UML sources using only my accepted review decisions: [list]. Keep justified existing elements and explain each
change.
Preserve the agreed scope and rule IDs. Return both updated PlantUML blocks and a concise change log. Flag any unresolved
issue.
Approved requirements: [paste]. Original sources: [paste]. My review decisions and evidence: [paste].
```

4 The two diagrams

Sequence diagram `models/sequence.puml`

- Every participant declared, every message named.
- A message naming the **catalog** or the **offerings**, then one naming **register** or **enrol**.
- The **duplicate** and **capacity** checks sit between the request and the creation — not only in the catalog view.
- Three guarded alternatives, written **without square brackets**:
  ```
  alt already enrolled
  else course full
  else not enrolled and places left
  ```
- A message that **creates** the Enrollment; the **confirmation** comes after it.
- Rejection branches return **course full** / **already enrolled** and nothing that creates, saves, updates or confirms.
- A `note` or a `group` containing the word **atomic**.

Class diagram `models/classes.puml` `(plural)`

- `Student`, `Course`, `CourseOffering`, `Enrollment`, spelled exactly. An extra class needs a `' why: <reason>` comment on the line above it.
- A student identifier, `Course` code and title, `CourseOffering` semester and **capacity**.
- Multiplicities at **both ends** of every association:
  ```
  Student "1" -- "0..*" Enrollment : holds >
  ```
- Each Enrollment links **one** student to **one** offering.
- Capacity and uniqueness constraints stated separately, in **notes** — multiplicity alone cannot express them.
- In `report.md` §4, read every association in both directions.

Renders. Export four images to `models/img/` under these names: `sequence`, `classes`, `original-sequence`, `original-classes` (`.png` or `.svg`). **Your final may be identical to the AI's original** if the critique found no genuine defect and your walkthroughs pass — then §7 says what you verified and why you kept the design. Do not invent an error to fill the log.

Be ready to explain, without the assistant: Where is the Enrollment created? What stops two students taking the last place? Why is uniqueness a note and not a multiplicity?

5 Validate: four cases, walked through your own diagram

Case	Starting state	Expected result
T1. Available	Capacity 2, enrolled 0. New student.	Accept. Create one enrollment. Count becomes 1.
T2. Last place	Capacity 2, enrolled 1. New student.	Accept. Count becomes 2. No places remain.
T3. Full	Catalog showed a place. Current count is 2 of 2.	Reject as full. No new record. Count stays 2.
T4. Duplicate	Capacity 2, enrolled 1. That student requests again.	Reject as duplicate. Count stays 1.

A walkthrough row (§6) gives the actual steps through your diagram, message by message, and the state change (10 words or more). The judgment starts with **Pass** or **Fail**, then the location: the message, guard or note where you saw it. **An honest Fail is a finding, not a penalty.** **Traceability (§5):** one row per rule R1–R5 — the criteria that cover it, the model evidence, the case that exercises it.

A worthless review decision (§7)	One worth something: the element, your decision, a reason of 8 words or more, the rule
"Sequence diagram — accepted — looks good — R4"	"Original: `confirm registration` sent before `countEnrollments` — moved after `createEnrollment`, success branch only — in T3 the student was told they were registered and then rejected as full — R4, R2"

6 Check, declare, submit

```
cd sis-02
python tests/check_sis.py             # 44 checks of SHAPE, never quality - copy its three numbers
git add . && git commit -m "sis-02: final models and report"
git rev-parse --short HEAD            # this hash goes in checker.commit in submission.yml
python tests/validate_submission.py   # fix anything it flags
git add submission.yml && git commit -m "sis-02: declaration" && git push
```

- **PASS** the check is satisfied · **FAIL** the thing is there but wrong · **ERROR** it cannot be checked at all (a file or a heading is missing). A fresh template prints `pass=5 fail=23 error=16`; that is the intended start.
- **Your numbers are re-run at the head of your pull request.** After your final run, change nothing in `report.md` or `models/` — or run the checker again and update the declaration. Numbers that do not match cost the whole workflow criterion.
- **A FAIL you report costs nothing extra:** list its ID under `known_fails` and give the reason in a §7 row.
- **No Python?** On GitHub: Code → Codespaces → Create codespace on `sis-02`, and run the same commands there.
- **Submit:** branch `sis-02` off `main`, folder `sis-02/` at the top of `se-practice`. One pull request `sis-02 → main`, title `SIS 02 - Course registration - <Surname Name>`, description in the five sections from `SETUP.md` §4 (all five headings are checked). **Leave it open, never merge it.** In Teams: + Add work → Link → the pull request URL, not the repository URL.

7 Grading — 2 points, 8 criteria × 0.25 (full · half · zero)

Criterion	Full marks when
Requirements baseline	Scope and assumptions written before Prompt A and kept in §10; US-01–03; AC-01–06 in Given / When / Then, each mapped to R1–R5; reviewed, nothing out of scope.
Sequence diagram	Catalog browsing, selection and registration; duplicate and capacity checks before creation, as one atomic operation; guarded success, full and duplicate paths; a rejection changes no state.
Class diagram	The four classes with identifiers and capacity; multiplicities at both ends; capacity and uniqueness constraints stated separately; extra classes justified.
Consistency & traceability	Same names and IDs in stories, criteria, diagrams and cases; technical lifelines explained; every rule traced to a criterion, model evidence and a case.
Walkthroughs & review decisions	T1–T4 each walked through the actual diagram: path, state change, verdict, location. Three substantive decisions, each with the element, the decision, a reason and the rule.
Explanation & reflection	Main design choices; how the two diagrams complement each other; why an activity diagram could help. A specific reflection: where AI helped, one decision of your own, one limitation.
AI evidence & source checks	Four complete exchanges with tool, model, date, purpose; Prompts A–D as given; originals kept separate from the finals; two modeling decisions checked in sources you actually read, exact locators.
Structure & workflow	Word counts in range or declared; `sis-02` branch, 3+ own commits, the first before Prompt A; open PR in the standard shape; `AI_USAGE.md`; `submission.yml` validating.

A FAIL you report and explain costs you nothing; one you hide costs the whole criterion.

Not accepted (0 for the SIS): a file upload instead of a pull-request link · a report that is not in `sis-02/report.md` · diagrams as images only, without PlantUML source · a merged PR, a repository link or a branch with no PR · fabricated prompts, responses, walkthroughs or references.